

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 13. Dictionary

Lecture: 141. Dictionary Methods

Section 1: Lecture Summary

This lecture focused on essential Python dictionary methods used to add, update, copy, and delete items in dictionaries. It began by discussing the `update` method, which merges one dictionary into another. The static method `fromkeys` was explained as a way to create new dictionaries from a sequence of keys with a default value. The `copy` method was introduced as a way to create a shallow copy of a dictionary, highlighting the difference between shallow and deep copies. Deletion methods were covered next: `pop` removes an entry with a given key and can accept a default return value if the key is missing; `popitem` deletes the most recently added key-value pair; and `clear` removes all items from a dictionary. Throughout the lecture, practical examples in Jupyter Notebook were provided to demonstrate these methods in action and their effects on dictionary contents.

Section 2: Key Concepts and Explanations

Dictionary Methods for Adding and Updating Items

- `update(dictionary)`

Adds key-value pairs from one dictionary to another, overwriting existing keys if necessary.

Example: `d1.update(d2)` merges `d2` into `d1`.

- `fromkeys(sequence, default=None)`

A static method called on the `dict` class itself, not on dictionary instances. Creates a new dictionary with keys from the provided sequence and all values set to the given default (default is `None` if not specified).

Example: `dict.fromkeys(['a', 'b', 'c'], 'unknown')` creates `{'a': 'unknown', 'b': 'unknown', 'c': 'unknown'}`.

Copying Dictionaries

- `copy()`

Returns a shallow copy of the dictionary. This means the new dictionary has its own key-value table, but the keys and values themselves are referenced (not deeply copied). Changes

to mutable values inside will reflect in both dictionaries, but adding/removing items affects only the copy.

Important distinction: shallow copy duplicates structure, not nested objects or referenced values.

Deleting Items from Dictionaries

- **pop(key, alternate_value=None)**

Removes the key-value pair with the specified key and returns its value. If the key is not found and the alternate value is provided, it returns that instead of raising an error. If the alternate value is not provided and the key is missing, a `KeyError` occurs.

Example: `d1.pop(5, "Missing")` returns `"Missing"` if key `5` does not exist.

- **popitem()**

Removes and returns the most recently inserted key-value pair as a tuple. Useful for stack-like operations on dictionaries.

- **clear()**

Removes all items from the dictionary, resulting in an empty dictionary `{}`.

Additional Notes

- Method `fromkeys` is a class-level method of `dict`, not instance method — useful before any dictionary exists.

- Understanding shallow vs. deep copy is critical when working with mutable objects inside dictionaries.

- Deletion methods modify the dictionary in place.

Section 3: Example Code and Use Cases

Example: update method

Output: `{1: 'One', 2: 'Two', 3: 'Three', 4: 'Four', 5: 'Five'}`

```
d1 = {1: 'One', 2: 'Two', 3: 'Three', 4: 'Four'}
d2 = {5: 'Five'}
d1.update(d2)
print(d1)
```

Updates dictionary `d1` by adding entries from `d2`.

Example: fromkeys method

Output: {1: None, 2: None, 3: None, 4: None}

Output: {1: 'unknown', 2: 'unknown', 3: 'unknown', 4: 'unknown'}

```
l1 = [1, 2, 3, 4]
d_new = dict.fromkeys(l1)
print(d_new)

d_new_with_default = dict.fromkeys(l1, "unknown")
print(d_new_with_default)
```

Creates a new dictionary with keys from list l1; all values set to None or a specified default.

Example: copy method (shallow copy)

Output: {1: 'One', 2: 'Two', 3: 'Three', 4: 'Four'}

```
d1 = {1: 'One', 2: 'Two', 3: 'Three', 4: 'Four'}
d2 = d1.copy()
print(d2)
```

Creates a shallow copy of d1 in d2.

Example: pop method with error and default

Output:

Two

{1: 'One', 3: 'Three', 4: 'Four'}

Pop with missing key without default raises KeyError

d1.pop(5) Would raise KeyError

Pop with missing key with default avoids error

Output: Missing

```
d1 = {1: 'One', 2: 'Two', 3: 'Three', 4: 'Four'}
value = d1.pop(2)
print(value)
print(d1)

missing_value = d1.pop(5, 'Missing')
print(missing_value)
```

Removes a key, returns value; shows handling of missing keys.

Example: popitem method

Output:

(4, 'Four')

{1: 'One', 2: 'Two', 3: 'Three'}

```
d1 = {1: 'One', 2: 'Two', 3: 'Three', 4: 'Four'}  
item = d1.popitem()  
print(item)  
print(d1)
```

Removes and returns the most recently added key-value pair.

Example: clear method

Output: {}

```
d1.clear()  
print(d1)
```

Deletes all entries in the dictionary.

Section 4: Key Takeaways

- The `update` method merges dictionaries, adding or overwriting keys.
- `fromkeys` is a class method creating a new dictionary from a sequence of keys with a uniform default value.
- `copy` produces a shallow copy, duplicating the dictionary structure but not nested mutable objects.
- `pop` removes and returns a key's value, with optional default to prevent errors on missing keys.
- `popitem` removes the most recently inserted item, returning it as a `(key, value)` tuple.
- `clear` empties the dictionary completely.
- Knowing how these dictionary methods behave helps efficiently manage dynamic dictionary contents in Python programs.